NESTLY · ENGINEERING DOCUMENTATION

Low-Level Design

Backend implementation · schema · concurrency · payments · workers · APIs

Contents

1 Implementation architecture and repository
2 Module boundaries
3 Request lifecycle, auth and RBAC
4 Domain model and order snapshots
5 PostgreSQL schema, constraints and indexes
6 Reservation and booking implementation
7 Payments and idempotency
8 Saga and compensation
9 Redis, queues and workers
10 REST API contract
11 Validation, errors, logging and security
12 Search and media
13 Configuration, Docker and AWS
14 Testing, concurrency and load testing
15 Failure recovery
16 Implementation order and definition of done
17 Complete implementation diagram

1. Implementation architecture and repository

The backend is JavaScript on Node.js and Express, organized by **business module** with shared infrastructure for PostgreSQL, Redis, SQS, configuration and logging. Each module follows the same layering.

```
nestly/
├── client/                     React SPA
├── server/src/
│   ├── config/                 env parsing, constants
│   ├── middleware/             requestId, logger, auth, rbac, validate, rateLimit, errorHandler
│   ├── infrastructure/         pg pool + withTx, redis client + lock, sqs, s3, opensearch, payment SDK
│   ├── modules/
│   │   ├── auth  users  properties  search  providers  services  events  packages
│   │   └── availability  reservations  orders  payments  reviews  notifications  admin
│   │       (each: routes → controller → validation → service → repository)
│   └── workers/                consumers + schedulers (own entrypoint, same codebase)
├── nginx/                      client reverse proxy config
├── load-tests/k6/              scenarios
├── infrastructure/aws/         IaC
├── docs/
└── docker-compose.yml
```

Layer	Responsibility	Must not
Route	Map method and path to middleware chain and controller	Contain logic
Controller	Translate HTTP to a service call and the result back to HTTP	Touch the database or Redis
Validation	Schema-validate body, params, query (rejects before any logic)	Be skipped for any mutation
Service	Business rules, transactions, orchestration, emits outbox events	Know about HTTP
Repository	Parameterized SQL only; accepts a transaction handle	Contain business rules

Workers run from the same codebase with a separate entrypoint, so services and repositories are reused and API and worker containers scale independently.

2. Module boundaries

Module	Owns	Depends on	Exposes to others
Auth	Login, tokens, password hashing	Users	verifyToken, issueToken
Users	user records, roles	—	getUser, roleOf
Properties	listing, images, amenities	Availability	getProperty, listByHost
Providers / Services	provider and service data	Availability	getService, listByProvider
Events / Packages	event intent; reusable service groups	Services	buildOrderItems
Availability	resource schedules, capacity	Properties, Services	isBookable (advisory)
Reservations	holds and reservation rows	PostgreSQL, Redis	hold, confirm, release, expire
Orders	order and order items, price snapshots	Reservations, Payments	createOrder, getOrder
Payments	payment state, transactions, webhooks	Payment provider	startPayment, applyEvent
Notifications	records and delivery	SQS	enqueue (via events)
Search	query API over the index	OpenSearch	search
Admin	moderation, audit	Domain modules	audit, moderate

Rule: no module reads or writes another module's tables. Cross-module effects use service calls (synchronous, same transaction where needed) or outbox events (asynchronous).

3. Request lifecycle, auth and RBAC

Middleware order (every request)

1. **requestId**: generate or accept a correlation ID; attach to logger and response headers.
2. **logger**: structured start/finish log with operation, status and duration.
3. **Helmet and CORS**: security headers; restrictive origin allow-list.
4. **rateLimit**: Redis counters keyed rate:{scope}:{window}.
5. **authN**: verify JWT signature, expiry, audience; load role.
6. **authZ (RBAC)**: check the role allowed for this route; then resource ownership in the service.
7. **validate**: schema-check body, params and query; reject unknown fields.
8. **controller → service**: business logic; invariants are re-checked inside transactions.
9. **errorHandler**: map errors to the standard error shape; never leak stack traces.

Capability	CUSTOMER	HOST	PROVIDER	ADMIN
Search and view catalog	✓	✓	✓	✓
Create and manage property	—	own	—	✓
Create and manage service	—	—	own	✓
Create event, order, hold, pay	own	—	—	✓
Cancel order	own	—	—	✓
Post review	eligible	—	—	—
Moderate, audit, payouts	—	—	—	✓

Frontend visibility is convenience only. Authorization is always enforced server-side, and ownership is checked in the service layer.

4. Domain model and order snapshots

- **Property:** host_id, title, description, location, capacity, pricing, status. Images and amenities are separate records.
- **Service:** provider_id, price, duration/capacity, status, and whether it is exclusive or capacity-based.
- **Event:** captures customer intent (date, guests, type). **Package:** a reusable set of services via package_services.
- **Checkout:** chosen property and services become **order_items with price snapshots** (unit price, currency, quantity, resource reference). Later catalog changes never alter an existing order.

Resource kind	Examples	Availability model	Enforced by
Exclusive	Property on a date range; a single photographer	One active reservation per resource and time range	Exclusion constraint
Capacity-based	Caterer taking up to N events per slot	reserved ≤ capacity per slot	Atomic conditional UPDATE

5. PostgreSQL schema, constraints and indexes

Table	Key fields and constraints
users	PK id; UNIQUE email; role; password_hash
properties / property_images / amenities	PK id; FK host_id; FK property_id
providers / services	providers: FK user_id (unique); services: FK provider_id, kind (exclusive|capacity)
events / packages / package_services	events: FK customer_id; package_services: unique (package_id, service_id)
availability / service_capacity	resource_id + slot; capacity and reserved with CHECK (reserved ≤ capacity)
orders / order_items	orders: FK user_id, status, indexes on (user_id, created_at), status; order_items: FK order_id, resource ref, price snapshot
reservations	FK order_item_id; resource_id, during, status, hold_expires_at; EXCLUDE constraint (below)
payments / payment_transactions	payments: FK order_id, provider reference; transactions: UNIQUE provider_event_id
idempotency_keys	PK (user_id, operation, idempotency_key); request_hash; status; expires_at
outbox / saga_instances	outbox: unpublished events; saga_instances: UNIQUE order_id, state, step, context
reviews / notifications / audit_logs / payouts	reviews: eligibility via completed booking; notifications: (user_id, status); audit_logs: (entity, created_at)

Reservations: overlap protection at the database

```
CREATE EXTENSION IF NOT EXISTS btree_gist;
CREATE TABLE reservations (
  id              uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  order_item_id   uuid NOT NULL REFERENCES order_items(id),
  resource_type   text NOT NULL CHECK (resource_type IN ('PROPERTY','SERVICE')),
  resource_id     uuid NOT NULL,
  during          tstzrange NOT NULL,
  status          text NOT NULL CHECK (status IN ('HELD','PAYMENT_PENDING','CONFIRMED','CANCELLED','EXPIRED')),
  hold_expires_at timestamptz,
  created_at      timestamptz NOT NULL DEFAULT now(),
  EXCLUDE USING gist (resource_id WITH =, during WITH &&)
    WHERE (status IN ('HELD','PAYMENT_PENDING','CONFIRMED'))
);
CREATE INDEX reservations_hold_sweep ON reservations (hold_expires_at) WHERE status = 'HELD';
```

Only active statuses participate in the constraint, so cancelled and expired rows free the slot. An overlapping insert fails with SQLSTATE **23P01**, which the service maps to RESOURCE_CONFLICT.

Supporting tables

```
CREATE TABLE idempotency_keys (
  user_id uuid NOT NULL, operation text NOT NULL, idempotency_key text NOT NULL,
  request_hash text NOT NULL,
  status text NOT NULL CHECK (status IN ('IN_PROGRESS','COMPLETED','FAILED')),
  response_reference text,
  created_at timestamptz NOT NULL DEFAULT now(), expires_at timestamptz NOT NULL,
  PRIMARY KEY (user_id, operation, idempotency_key));

CREATE TABLE payment_transactions (
  id uuid PRIMARY KEY, payment_id uuid NOT NULL REFERENCES payments(id),
  provider_event_id text NOT NULL UNIQUE,
  type text NOT NULL, amount_minor bigint, created_at timestamptz DEFAULT now());

CREATE TABLE outbox (
  id uuid PRIMARY KEY, aggregate_id uuid NOT NULL, event_type text NOT NULL,
  payload jsonb NOT NULL, created_at timestamptz DEFAULT now(), published_at timestamptz);

CREATE TABLE saga_instances (
  id uuid PRIMARY KEY, order_id uuid NOT NULL UNIQUE, state text NOT NULL, step int NOT NULL,
  context jsonb NOT NULL, updated_at timestamptz NOT NULL DEFAULT now());
```

6. Reservation and booking implementation

Transaction sequence: validate order ownership → acquire coordination lock → begin transaction → expire stale holds for the resource → insert reservation → write outbox event → commit → release lock → continue to payment.

Lock helper (Redis)

```
// infrastructure/redis/lock.js
const RELEASE = `if redis.call('get',KEYS[1])==ARGV[1]
                 then return redis.call('del',KEYS[1]) else return 0 end`;

async function acquire(key, ttlMs) {
  try {
    const token = crypto.randomUUID();
    const ok = await redis.set(key, token, 'PX', ttlMs, 'NX');
    return ok ? { status: 'acquired', token } : { status: 'busy' };
  } catch { return { status: 'unavailable' }; }          // Redis down
}
const release = (key, token) => redis.eval(RELEASE, 1, key, token);  // compare-and-delete
```

Reserve (service layer)

```
async function hold(item, slot, holdMs) {
  const key  = `lock:resource:${item.resourceId}:${slot}`;
  const lock = await acquire(key, 5000);
  if (lock.status === 'busy') throw new AppError('RESOURCE_BUSY', 409);   // fast reject
  // 'unavailable' => continue without lock: correctness comes from the DB below
  try {
    return await db.withTx(async (tx) => {
      await repo.expireStaleHolds(tx, item.resourceId);   // expired HELD rows => EXPIRED
      const r = await repo.insertHeld(tx, item, holdMs);  // may raise 23P01 (overlap)
      await outbox.add(tx, 'reservation.held', r.id);
      return r;
    });
  } catch (e) {
    if (e.code === '23P01') throw new AppError('RESOURCE_CONFLICT', 409);
    throw e;
  } finally {
    if (lock.token) await release(key, lock.token);
  }
}
```

Concern	Implementation
Lock TTL shorter than transaction	Safe: the exclusion constraint still rejects the overlap. The lock reduces contention; it is not the guard.

Concern	Implementation
Hold expiry	hold_expires_at is authoritative. Two paths reconcile: lazy expiry inside the booking transaction, and a sweeper running every ~30 s that sets HELD rows past expiry to EXPIRED and emits events.
Redis hold key	Optional fast-path hint (hold:{resource}:{slot}:{id} with TTL). Never trusted alone; key expiry does not change PostgreSQL.
Capacity-based service	UPDATE service_capacity SET reserved = reserved + $q WHERE service_id=$1 AND slot=$2 AND reserved + $q <= capacity; zero rows updated means full.
Confirm	After payment success: UPDATE reservations SET status='CONFIRMED' WHERE id=$1 AND status IN ('HELD','PAYMENT_PENDING') AND hold_expires_at > now() — zero rows triggers the compensation path.

7. Payments and idempotency

State	Meaning	Allowed transitions (guarded by current state)
CREATED	Record exists	→ PENDING
PENDING	Waiting to proceed	→ PROCESSING, FAILED
PROCESSING	Provider operation in flight	→ SUCCESS, FAILED; stays if ambiguous until reconciled
SUCCESS	Payment confirmed	→ REFUNDED
FAILED	Terminal failure	none
REFUNDED	Successful payment refunded	none

Transitions are applied by one function using a guarded update, for example `UPDATE payments SET status=$new WHERE id=$1 AND status = ANY($allowedFrom)`. Zero rows updated means the transition already happened or is illegal; the caller treats it as a no-op.

Idempotent payment request

1. Require the **Idempotency-Key** header; respond 400 if absent.
2. Hash the canonical request body. Insert (user_id, operation, key) as IN_PROGRESS with ON CONFLICT DO NOTHING.
3. Inserted: execute the operation, store the response reference, mark COMPLETED.
4. Conflict and same hash: COMPLETED returns the stored result (200); IN_PROGRESS returns 409 REQUEST_IN_PROGRESS so the client retries.
5. Conflict and different hash: 422 IDEMPOTENCY_KEY_REUSED.
6. Send the provider a key derived from the payment ID so provider-side retries are safe too; expire rows after roughly 24 hours.

Webhook handler

1. Read the **raw body** (the webhook route uses a raw-body parser) and verify the provider signature; reject with 401 on failure.
2. In one transaction: INSERT into payment_transactions with provider_event_id (UNIQUE) ON CONFLICT DO NOTHING.
3. No row inserted means a duplicate: return 200 and stop.
4. Apply the guarded state transition, write outbox events (order confirm or compensation), commit, return 200 quickly.

Reconciler: a scheduler selects payments in PROCESSING older than a threshold, asks the provider for the status by reference, and applies the same transition function. This resolves timeouts and missed webhooks.

8. Saga and compensation

The **SagaOrchestrator** lives in the Orders module. Its progress is persisted in *saga_instances* (state, step, context), so a process crash never loses the workflow.

Step	Forward action	Compensation	Idempotent by
1	Hold property (reservations.hold)	release(reservationId)	guarded status update
2	Hold each service	release each	guarded status update
3	Start payment (Idempotency-Key)	refund or void	provider key + payment state guard
4	Confirm reservations; order CONFIRMED	retry; if impossible, refund + release	conditional UPDATE

- Failure at step N runs compensations for steps N−1 … 1 in reverse order. Compensations are retried with backoff until they succeed; each is safe to repeat.
- **Recovery:** on startup and every minute, a saga-recovery scheduler resumes instances that are not terminal and were not updated recently.
- **Outbox:** business changes and their events are written in one transaction; the outbox publisher polls unpublished rows, sends to SQS and marks them published. Delivery is at-least-once, so consumers dedupe.
- Payment order (authorize-then-capture versus pay-then-confirm) is an open business decision (D-3); step 3 and its compensation change accordingly.

9. Redis, queues and workers

Redis key	Purpose	TTL
`cache:property:{id}`	Property detail cache	minutes
`cache:search:{hash}`	Short-lived search cache	seconds–1 min
`lock:resource:{id}:{slot}`	Contention coordination (owner token)	~5 s
`hold:{resource}:{slot}:{id}`	Optional hold hint; PostgreSQL is authoritative	hold duration
`rate:{scope}:{window}`	Rate-limit counters	window

Queue	Producer	Consumer	Idempotency key
`notifications`	outbox publisher	Notification worker	eventId
`provider-actions`	outbox publisher	Provider worker	orderItemId + action
`analytics`	API / outbox	Analytics worker	eventId (loss-tolerant)
`search-index`	outbox publisher	Indexer	document ID + version

```
// message envelope
{ "eventId": "uuid", "eventType": "reservation.held", "occurredAt": "ISO-8601",
  "aggregateId": "uuid", "version": 1, "payload": { } }
```

- Long-poll SQS; set the visibility timeout above worst-case handler time; extend it for slow jobs.
- Delete a message only after successful processing. Failures return to the queue; after maxReceiveCount (for example 5) SQS moves it to the queue's DLQ.
- Handlers dedupe on eventId (or a natural key) because delivery is at-least-once. Indexer upserts are naturally idempotent.
- Alarm on any DLQ depth above zero and on oldest-message age.
- **Schedulers** (same codebase): outbox publisher, hold sweeper, payment reconciler, saga recovery, idempotency-key cleanup. Use a single-runner guard (advisory lock) so multiple worker instances do not

double-run them.

10. REST API contract

All routes are under **/api/v1**. Rows marked * are additions needed to complete the journey.

Method	Endpoint	Auth	Success	Notable errors
POST	`/auth/register`	Public	201	409 EMAIL_TAKEN
POST	`/auth/login`	Public	200	401 INVALID_CREDENTIALS
GET	`/properties`	Public	200	—
GET	`/properties/:id`	Public	200	404
POST	`/properties`	HOST	201	400, 403
PUT	`/properties/:id *`	HOST (own)	200	403, 404
GET	`/properties/:id/availability *`	Public	200 (indicative)	404
GET	`/services`	Public	200	—
POST	`/services *`	PROVIDER	201	400, 403
POST	`/events`	CUSTOMER	201	400
POST	`/orders`	CUSTOMER	201	400, 422 ITEM_INVALID
GET	`/orders/:id *`	Owner/Admin	200	403, 404
POST	`/orders/:id/hold`	CUSTOMER	200	409 RESOURCE_CONFLICT / RESOURCE_BUSY
POST	`/payments`	CUSTOMER + Idempotency-Key	202 (200 on replay)	409 REQUEST_IN_PROGRESS, 410 HOLD_EXPIRED, 422 IDEMPOTENCY_KEY_REUSED
GET	`/payments/:id *`	Owner/Admin	200	404
POST	`/payments/webhook`	Provider signature	200	401 INVALID_SIGNATURE
POST	`/orders/:id/cancel`	Owner/Admin	200	409 NOT_CANCELLABLE
POST	`/reviews`	Eligible customer	201	403 NOT_ELIGIBLE

Every mutation documents request body, response body, status codes, auth, idempotency and conflict behavior. Errors follow one shape, with a stable machine-readable code:

```
{ "error": { "code": "RESOURCE_CONFLICT", "message": "Resource is no longer available",
             "requestId": "req_...", "details": {} } }
```

Code	HTTP	Client should
`RESOURCE_CONFLICT`	409	Choose another slot or resource; do not retry
`RESOURCE_BUSY`	409	Retry shortly with jitter
`HOLD_EXPIRED`	410	Restart checkout
`REQUEST_IN_PROGRESS`	409	Retry the same request with the same key
`IDEMPOTENCY_KEY_REUSED`	422	Use a new key for a different request
`VALIDATION_ERROR`	400	Fix input per details
`RATE_LIMITED`	429	Back off using Retry-After

11. Validation, errors, logging and security

- **Validation:** one schema per route (body, params, query); unknown fields rejected; money handled as integer minor units.
- **Errors:** a single error middleware; expected failures are AppError with a code and HTTP status; unexpected errors become 500 INTERNAL with the request ID only. Never expose stack traces, SQL internals or secrets.
- **Logging:** structured JSON with requestId, operation, userId (where safe), orderId/resourceId, duration, status and dependency outcome. Never log tokens, passwords or card data.
- **Security:** bcrypt/argon2 password hashing, JWT verification with short expiry, RBAC, parameterized queries, Helmet, restrictive CORS, rate limits, Secrets Manager, least-privilege IAM.

12. Search and media

Search indexing

1. A catalog change writes an outbox row in the same transaction as the business change.
2. The outbox publisher sends it to the search-index queue.
3. The Indexer loads the current record from PostgreSQL and upserts the OpenSearch document by ID with a version, so replays and out-of-order messages are harmless.
4. The Search API queries OpenSearch and returns results marked indicative; checkout revalidates in PostgreSQL. A full reindex from PostgreSQL rebuilds the index at any time.

Media upload

1. Client requests an upload URL (auth, size and content-type limits enforced).
2. API returns a presigned S3 PUT URL with a short expiry and a generated object key.
3. Browser uploads directly to S3; the client then confirms, and the API stores metadata (key, owner, property) in PostgreSQL.
4. Reads go through CloudFront. Large files never pass through the API.

13. Configuration, Docker and AWS

Docker Compose runs the React client, Node API, PostgreSQL, Redis and optional workers for local development. Use multi-stage builds; Nginx serves and reverse-proxies the client container only.

Variable	Purpose
`NODE_ENV, PORT, LOG_LEVEL`	Runtime mode, listen port, verbosity
`DATABASE_URL, DB_POOL_MAX`	PostgreSQL connection and pool size (instances × pool must stay below DB limit)
`REDIS_URL`	ElastiCache endpoint
`JWT_ACCESS_SECRET, JWT_TTL`	Token signing and expiry (from Secrets Manager in AWS)
`AWS_REGION, S3_BUCKET, CLOUDFRONT_DOMAIN`	Media configuration
`SQS_QUEUE_URL_*`	One URL per queue
`OPENSEARCH_ENDPOINT`	Search cluster
`PAYMENT_SECRET, PAYMENT_WEBHOOK_SECRET`	Provider API key and webhook signing secret
`HOLD_TTL_SECONDS, LOCK_TTL_MS`	Business and lock timing

In AWS use IAM roles instead of static credentials. Deployment: build image → push to ECR → deploy to EC2 (or ECS) behind the ALB; RDS, ElastiCache, SQS, S3/CloudFront, CloudWatch and Secrets Manager as in the HLD.

Health endpoints: `/healthz` (process alive) and `/readyz` (database reachable).

14. Testing, concurrency and load testing

Test	Expected behavior
Two users, same property and date	Exactly one confirmation; the other gets RESOURCE_CONFLICT
Same test with Redis disabled	Still exactly one confirmation (constraint holds)
Lock TTL expires mid-transaction	No double booking
Duplicate payment request	One logical payment; second call returns the original
Same key, different body	422 IDEMPOTENCY_KEY_REUSED
Duplicate webhook	No duplicate side effect
Expired hold	Resource bookable by others; confirm fails and compensates
Service fails in Saga	Earlier holds released; nothing charged
Crash mid-Saga	Recovery resumes or compensates
Worker failure	Retried, then DLQ
Redis miss / outage	DB read and cache populate / safe fallback
Search stale	Checkout still validates against PostgreSQL

k6 scenarios: normal search, burst search, same-resource contention, duplicate payment, queue stress.
Measure: RPS, P50/P95/P99, error rate, DB connections and CPU, cache hit ratio, queue depth. Only report numbers that were measured.

15. Failure recovery

Failure	Recovery
API process crash	ALB health check removes it; replacement starts; no local durable state
Database transient failure	Bounded retry for idempotent operations, otherwise fail the request with 503
Redis failure	Cache reads fall through; booking proceeds with database-only locking
SQS duplicate delivery	Idempotent consumer
Outbox publisher crash	Unpublished rows remain; next run publishes them
Payment ambiguity	Reconciler queries provider by reference and applies transition
Search unavailable	Degrade independently; rebuild index from PostgreSQL
Bad deployment	Health check fails rollout; roll back to previous image; migrations are backward-compatible

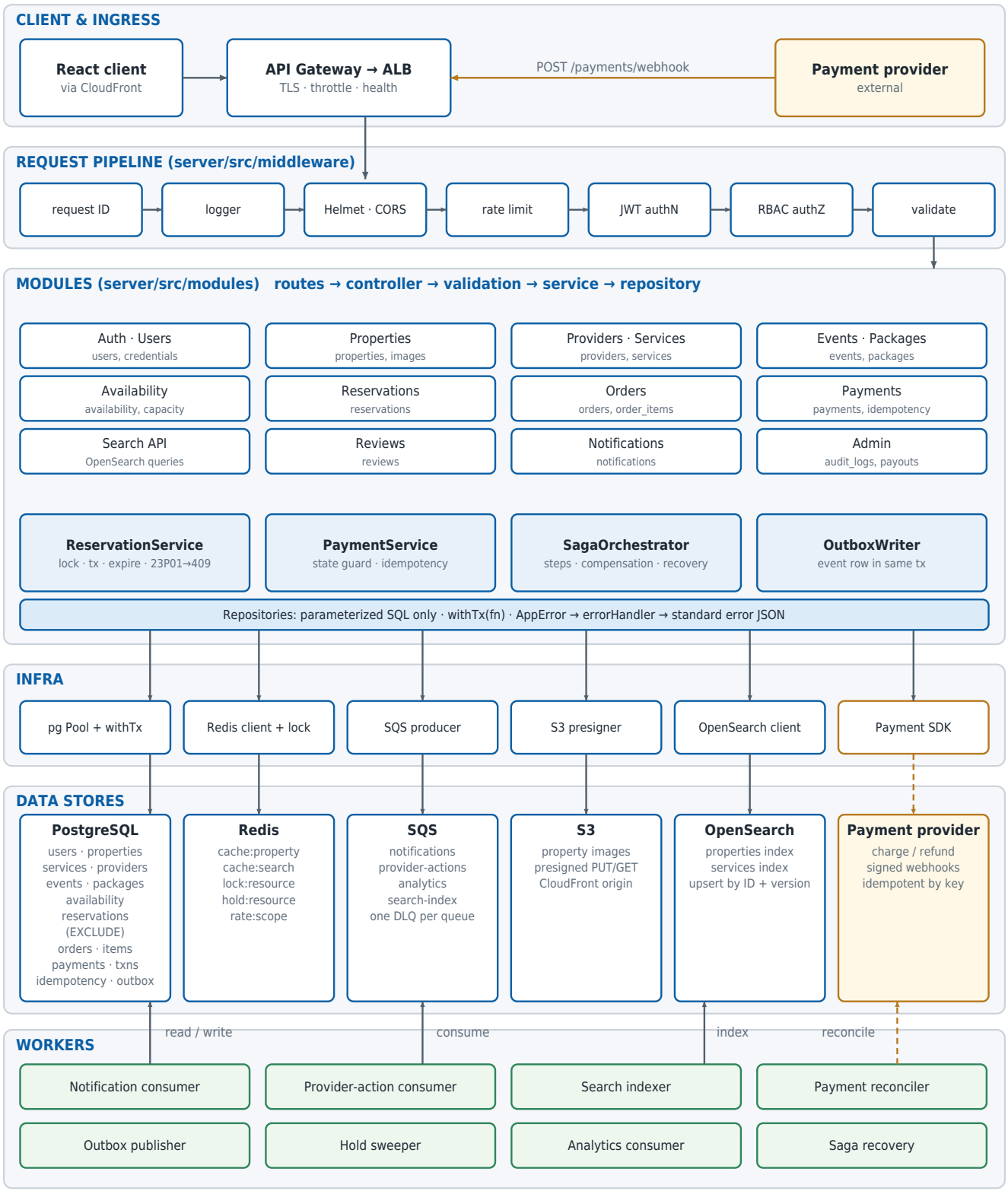
16. Implementation order and definition of done

#	Deliverable	Done when
1	Repository, config, Docker	docker compose up runs API, DB, Redis
2	PostgreSQL schema and migrations	Constraints and indexes applied; migrations versioned
3	Auth and RBAC	Role checks tested per route
4	Marketplace modules	CRUD for properties, services, events, packages
5	Availability and reservations	Overlap rejected by the constraint
6	Redis cache, holds, locks	Contention test passes with and without Redis
7	Orders	Price snapshots stored; ownership enforced
8	Payments and idempotency	Duplicate requests and webhooks are no-ops
9	Saga	Failure injection leaves no orphan holds
10	SQS and workers	Retry then DLQ demonstrated
11	S3 and OpenSearch	Direct upload and stale-tolerant search
12	AWS deployment	Runs behind ALB in private subnets
13	Observability	Dashboards and alarms fire on injected faults
14	Concurrency and k6 tests	Results recorded with measured numbers
15	Documentation	Docs match the implementation

Definition of done: the customer journey works end to end; concurrent booking is correct; payment retries are safe; Saga compensation works; workers retry and dead-letter safely; media uses object storage; Docker and AWS deployment work; logs and metrics expose failures; documentation matches the implementation.

17. Complete implementation diagram

One view of the implementation: request pipeline, modules and core services, infrastructure clients, data stores and workers.



HOW TO READ THIS

Every mutation: pipeline → controller → service → transaction; invariants are re-checked in PostgreSQL, never trusted from cache.
Booking: Redis lock (optional) → tx: expire stale holds → INSERT reservation (EXCLUDE) → outbox → commit → release lock.
Async: outbox publisher → SQS → consumers (idempotent, retry, DLQ). Solid = internal · dashed amber = external provider.